

Генерация кода

Теория автоматов и формальных языков

Лекция №14

Структура лекции

1. Разделение компилятора на интерфейсную и внутреннюю части.
2. Преобразование атомов в инструкции.
3. Однопроходные и многопроходные компиляторы.
4. Распределение регистровой памяти.

Архитектура компьютера

- До этого момента мы игнорировали архитектуру машины, для которой строим компилятор, то есть целевую машину.
- Под архитектурой мы подразумеваем определение центрального процессора компьютера с точки зрения программиста на машинном языке.
- Спецификации операций с набором инструкций, форматы инструкций, режимы адресации, форматы данных, регистры ЦП, инструкции ввода / вывода и т. д. - все это составляет то, что иногда называют традиционной архитектурой машинного языка (чтобы отличить ее от архитектуры уровня микропрограммирования, которую используют многие компьютеры).

Этап генерации кода

- После того, как все эти параметры четко и точно определены, мы можем завершить компилятор, выполнив этап генерации кода.
- Это фаза, которая принимает в качестве входных данных синтаксические деревья или поток атомов, выдаваемые фазой синтаксиса, и создает в качестве выходных данных программу объектного языка в виде двоично-кодированных инструкций в надлежащем формате.
- Основная цель генератора кода - преобразовать атомы или синтаксические деревья в инструкции.

Этап генерации кода

Многие проектировщики компиляторов рассматривают построение компиляторов как состоящую из двух логических частей задачу — построение внешнего интерфейса и внутреннего интерфейса.

Внешний интерфейс состоит из лексического и синтаксического анализа и не зависит от машины.

Бэкэнд состоит из генерации кода и оптимизации и зависит от машины, поэтому здесь начинается обсуждение скрытых или машинно-зависимых фаз компилятора.

Разделение компилятора на интерфейсную и внутреннюю части

- Это разделение на интерфейсную и внутреннюю части упрощает работу двумя способами при создании компиляторов для новых машин или новых языков.
- Во-первых, если мы реализуем компилятор для новой машины, и у нас уже есть компиляторы для нашей старой машины, все, что нам нужно сделать, это написать скрытую часть, поскольку интерфейс не зависит от машины.
- Наша жизнь также упрощается при создании компилятора для нового языка программирования на существующем компьютере. В этом случае мы можем использовать бэкенд, уже написанный для нашего существующего компилятора.

Преобразование атомов в инструкции

- Если мы временно проигнорируем проблему прямых ссылок (инструкций `Jump` или `Branch`), процесс преобразования атомов в инструкции будет относительно простым.
- По большей части все, что нам нужно, - это какой-то вариант `case`, `switch` или множественная ветвь назначения в зависимости от класса транслируемого атома.
- Каждый атом приведет к отдельной инструкции или последовательности инструкций.
- Если ЦП целевой машины требует, чтобы все арифметические операции выполнялись в регистрах, то пример преобразования атома `ADD` будет таким, как показано ниже на рисунке

Преобразование атомов

- атом ADD транслируется в инструкцию LOD (загрузить в регистр), за которой следует инструкция ADD, за которой следует инструкция STO (сохранить регистр в память).

```
(ADD, A, B, T1)  →      LOD   R1, A
                      ADD    R1, B
                      STO    R1, T1
```

Figure 6.3 Translation of an ADD Atom to Instructions

Преобразование атомов

- Большинство классов атомов будут реализованы аналогичным образом.
- Атомы условного перехода (в наших примерах называемые атомами **TST**) обычно реализуются как Load, Compare и Branch (переход), в зависимости от архитектуры целевой машины.
- Атом **MOV** (перемещение данных из одной ячейки памяти в другую) может быть реализован как Инструкция MOV (move), если это разрешено архитектурой целевой машины; в противном случае используется Load, за которым следует Store.

Адреса операндов

- Адреса операндов, которые появляются в атомах, должны быть соответствующим образом закодированы в формат команд целевой машины.
- Например, многие целевые машины требуют, чтобы к операциям обращались с помощью базового регистра и смещения от содержимого базового регистра.
- В этом случае генератор кода должен знать предполагаемое содержимое базового регистра и вычислять смещение, чтобы получить желаемый адрес операнда.
- Например, если мы знаем, что конкретный операнд находится в ячейке памяти 1E (шестнадцатеричный), а содержимое базового регистра - 10 (шестнадцатеричное), то смещение будет 0E, потому что $10 + 0E = 1E$.

Сравнение однопроходных и многопроходных компиляторов

- Есть несколько различных подходов к проектированию этапа генерации кода.
- Разница между этими подходами обычно характеризуется количеством проходов, которые выполняются над входным файлом.
- Для простоты предположим, что входной файл представляет собой файл атомов.
- Генератор кода, который сканирует этот файл атомов один раз, называется **однопроходным генератором кода**, а генератор кода, который сканирует его несколько раз называется **многопроходным генератором кода**.

Сравнение однопроходных и многопроходных компиляторов

- Наиболее серьезная проблема, связанная с принятием решения о том, использовать ли генератор кода с одним или несколькими проходами, связана с переходами вперед.
- Когда встречаются атомы, могут быть сгенерированы инструкции, а генератор кода поддерживает счетчик адресов памяти и программный счетчик.
- Когда встречается атом метки LABEL, этому атому присваивается значение в памяти (таблица меток записывается с адресом памяти, назначаемым каждой метке).
- Если встречается атом JUMP с адресом назначения, который находится выше в памяти, чем инструкция Jump (т.е. переход вперед), метка, к которой он переходит, еще не встречена, и будет невозможно сгенерировать инструкцию Jump полностью в это время.

Проблемы однопроходных компиляторов

- Пример этой ситуации показан на рисунке, где переход к метке L1 не может быть сгенерирован, потому что в то время, когда встречается атом JMP, генератор кода не обнаружил определение метки L1, которое будет иметь значение 9.

<u>Atom</u>	<u>Location</u>	<u>Instruction</u>
(ADD, A, B, T1)	4	LOD R1, A
	5	ADD R1, B
	6	STO R1, T1
(JMP, L1)	7	CMP 0, 0, 0
	8	JMP ?
(LBL, L1)		(L1 = 9)

Figure 6.4 Problem in Generating a Jump to a Forward Destination

Проблемы однопроходных компиляторов

- Есть два основных способа решить проблему прыжков вперед.
- Компиляторы с одним проходом решают эту проблему, сохраняя таблицу инструкций *Jump*, которые имеют прямое назначение.
- Каждая инструкция перехода с прямой ссылкой генерируется не полностью (без адреса назначения) при обнаружении, и каждая также вводится в таблицу исправлений вместе с меткой, на которую выполняется переход. При обнаружении каждого определения метки оно вводится в таблицу меток (*Fixup table*) вместе со значением адреса.

Проблемы однопроходных компиляторов

- Когда все атомы будут прочитаны, все атомы меток будут определены, и в это время генератор кода может повторно посетить все инструкции Jump в таблице Fixup и заполнить их адреса назначения.

<u>Atom</u>	<u>Loc</u>	<u>Instruction</u>	<u>Fixup Table</u>		<u>Label Table</u>	
			<u>Loc</u>	<u>Label</u>	<u>Label</u>	<u>Value</u>
(ADD, A, B, T1)	4	LOD R1, A				
	5	ADD R1, B				
	6	STO R1, T1				
(JMP, L1)	7	CMP 0, 0, 0				
	8	JMP 0	8	L1		
(LBL, L1)					L1	9
...						
EOF						
	8	JMP 9				

Figure 6.5 Use of the Fixup Table and Label Table in a Single Pass Code Generator for Forward Jumps

Многопроходные генераторы

- Многопроходные генераторы кода не требуют таблицы исправлений. В этом случае первый проход генератора кода ничего не делает, кроме построения таблицы меток, сохраняя адрес памяти для каждой метки.
- Затем, во втором проходе, все метки будут определены, и каждый раз, когда будет обнаружен переход, его метка назначения будет преобразована в таблице с назначенным адресом памяти.

Многопроходные генераторы

Begin First Pass:

<u>Atom</u>	<u>Loc</u>	<u>Instruction</u>	<u>Label Table</u>	
			<u>Label</u>	<u>Value</u>
(ADD, A, B, T1)	4-6			
(JMP, L1)	7-8			
(LBL, L1)			L1	9
...				
EOF				

Begin Second Pass:

<u>Atom</u>	<u>Loc</u>	<u>Instruction</u>
(ADD, A, B, T1)	4	LOD R1, A
	5	ADD R1, B
	6	STO R1, T1
(JMP, L1)	7	CMP 0, 0, 0
	8	JMP 9
(LBL, L1)		
...		
EOF		

Figure 6.6 Forward Jumps Handled by a Multiple Pass Code Generator

Примеры трансляции выражений

- `while (i<=x) { x = x+2; i = i*3; }`

где 0 и 1 – ячейки памяти.

- Набор атомов:

```
(LBL, L1)
(MOV, 1,, T1)
(TST, i, x,, 4, L2)           // Is i<=x ?
(MOV, 0,, T1)
(LBL, L2)
(TST, T1, 0,, 1, L3)         // Branch if T1 is false
(ADD, x, 2, x)
(MUL, i, 3, i)
(JMP, L1)                     // Repeat the loop
(LBL, L3)                     // End of loop
```

(1) Single Pass

<u>Atom</u>	<u>Loc</u>	<u>Instruction</u>	<u>Fixup Table</u>		<u>Label Table</u>	
			<u>Loc</u>	<u>Label</u>	<u>Label</u>	<u>Value</u>
(LBL, L1)					L1	2
(MOV, 1,, T1)	2	LOD R1,1				
	3	STO R1,T1				
(TST,i,x,,4,L2)	4	LOD R1,i				
	5	CMP R1,x,4				
	6	JMP 0	6	L2		
(MOV, 0,, T1)	7	LOD R1,0				
	8	STO R1,T1				
(LBL, L2)					L2	9
(TST,T1,0,,1,L3)	9	LOD R1,T1				
	10	CMP R1,0,1				
	11	JMP 0	11	L3		
(ADD, x, 2, x)	12	LOD R1, x				
	13	ADD R1,='2'				
	14	STO R1,x				
(MUL, i, 3, i)	15	LOD R1,i				
	16	MUL R1,='3'				
	17	STO R1,i				
(JMP, L1)	18	CMP 0,0,0				
	19	JMP 2				
(LBL, L3)					L3	20
...						
	6	JMP 9				
	11	JMP 20				

(2) Multiple passes

			Label Table	
<u>Atom</u>	<u>Loc</u>	<u>Instruction</u>	<u>Label</u>	<u>Value</u>
Begin First Pass:				
(LBL, L1)			L1	2
(MOV, 1, , T1)	2-3			
(TST, i, x, , 4, L2)	4-6			
(MOV, 0, , T1)	7-8			
(LBL, L2)			L2	9
(TST, T1, 0, , 1, L3)	9-11			
(ADD, x, 2, x)	12-14			
(MUL, i, 3, i)	15-17			
(JMP, L1)	18-19			
(LBL, L3)			L3	20

Begin Second Pass:

<u>Atom</u>	<u>Loc</u>	<u>Instruction</u>
(LBL, L1)		
(MOV, 1, , T1)	2	LOD R1,1
	3	STO R1,T1
(TST,i,x,,4,L2)	4	LOD R1,i
	5	CMP R1,x,4
	6	JMP 9
(MOV, 0, , T1)	7	LOD R1,0
	8	STO R1,T1
(LBL, L2)		
(TST,T1,0,,1,L3)	9	LOD R1,T1
	10	CMP T1,0
	11	JMP 20
(ADD, x, 2, x)	12	LOD R1,x
	13	ADD R1,='2'
	14	STO R1,x
(MUL, i, 3, i)	15	LOD R1,i
	16	MUL R1,='3'
	17	STO R1,i
(JMP, L1)	18	CMP 0,0,0
	19	JMP 2
(LBL, L3)		

Распределение регистровой памяти

- Распределение регистров - это процесс присвоения цели конкретному регистру или привязка регистра к переменной программиста или компилятора, так что для определенного диапазона или объема инструкций этот регистр имеет указанное назначение и не используется ни для каких других целей.
- Генератор кода должен поддерживать информацию о том, какие регистры используются для каких целей и какие регистры доступны для повторного использования.
- Основная цель при распределении регистров - максимальное использование регистров ЦП и минимизация ссылок на ячейки памяти.

Задача распределения регистровой памяти

- $A = B + C * D ;$
- $B = A - C * D ;$

Simple Register Allocation

```
LOD    R1, C
MUL     R1, D
STO     R1, Temp1
LOD     R1, B
ADD     R1, Temp1
STO     R1, A
LOD     R1, C
MUL     R1, D
STO     R1, Temp2
LOD     R1, A
SUB     R1, Temp2
STO     R1, B
```

Smart Register Allocation

```
LOD     R1, C
MUL     R1, D           C*D
LOD     R2, B
ADD     R2, R1          B+C*D
STO     R2, A
SUB     R2, R1          A-C*D
STO     R2, B
```

Figure 6.7 Register Allocation, Simple and Smart, for a Two Statement Program

Назначение регистров с использованием сбалансированного дерева разбора

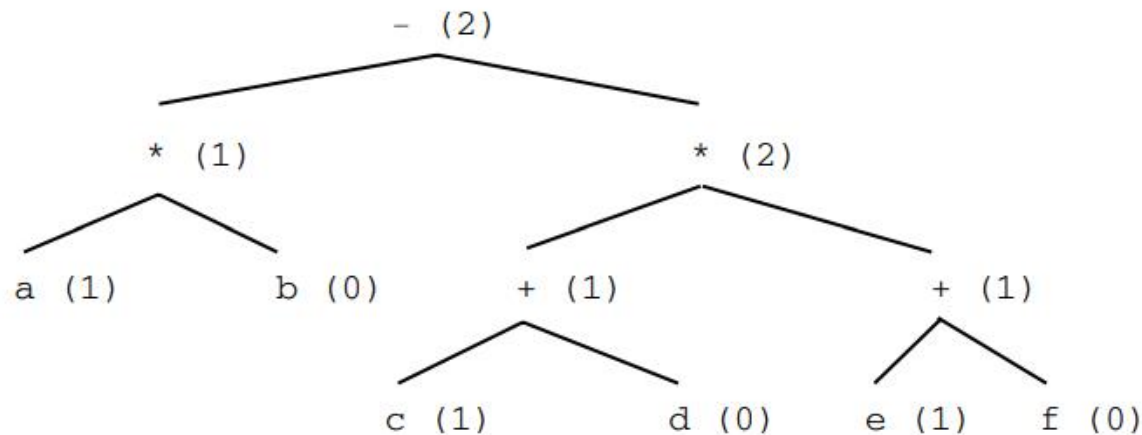


Figure 6.8 Weighted Syntax Tree for $a*b - (c+d)*(e+f)$, with Weights Shown in Parentheses

LOD	R1, c	
ADD	R1, d	$R1 = c + d$
LOD	R2, e	
ADD	R2, f	$R2 = e + f$
MUL	R1, R2	$R1 = (c+d) * (e+f)$
LOD	R2, a	
MUL	R2, b	$R2 = a * b$
SUB	R2, R1	$R2 = a*b - (c+d)*(e+f)$

Figure 6.9 Code Generated for $a*b - (c+d)*(e+f)$, Using Figure 6.8

Назначение регистров с использованием сбалансированного дерева разбора

- Эта проблема имела бы более интересное решение, если бы выражение имело вид:

$$e + f - (c + d) * (e + f)$$

- из-за повторяющегося подвыражения $(e + f)$.
- Если бы значение $(e + f)$ было оставлено в регистре, его не нужно было бы пересчитывать. Существуют алгоритмы, которые решают такие проблемы, но они касаются оптимизации процесса компиляции.